

1. AUTHENTICATION & AUTHORIZATION

Required Headers for Authenticated Requests:

text

Accept: application/json

Authorization: Bearer {access_token}

Registration

Endpoint: POST /api/register

Content-Type: multipart/form-data

Request Body:

text

phone: string (required, unique)

password: string (required, confirmed)

first_name: string (required)

last_name: string (required)

avatar: file (required, image)

profile_picture: file (required, image)

Success Response (200):

json

```
{
  "message": "Registered. Await admin approval."
}
```

Login

Endpoint: POST /api/login

Content-Type: application/json

Request Body:

json

```
{
  "phone": "1234567890",
  "password": "password123"
}
```

```
}
```

Success Response (200):

json

```
{
  "message": "Logged in successfully",
  "user": {
    "id": 1,
    "phone": "1234567890",
    "created_at": "2024-12-11"
  },
  "token": "1|abcdefghijklmnopqrstuvwxyz123456"
}
```

Logout

Endpoint: POST /api/logout

Headers: Authorization: Bearer {token}

Success Response (200):

json

```
{
  "message": "Logged out successfully"
}
```

2. USER MANAGEMENT

Update Profile

Endpoint: POST /api/profile/update

Headers: Authorization: Bearer {token}

Content-Type: multipart/form-data

Request Body (all fields optional):

text

```
phone: string
password: string
first_name: string
last_name: string
birth_date: date (YYYY-MM-DD)
avatar: file (image)
profile_picture: file (image)
```

Success Response (200):

json

```
{
  "message": "profile updated"
}
```

Get Profile

Endpoint: GET /api/profile/show

Headers: Authorization: Bearer {token}

Success Response (200):

json

```
{
  "first_name": "John",
  "last_name": "Doe",
  "profile_picture": "http://localhost/storage/Profile_Pictures/filename.jpg",
  "avatar": "http://localhost/storage/Avatars/filename.jpg",
  "birth_date": "1990-01-01",
  "phone": "1234567890"
}
```

3. CITY & NEIGHBORHOOD DATA

Get All Cities

Endpoint: GET /api/cities

Authentication: Not Required

Success Response (200):

json

```
[
  {
    "name": "Damascus",
    "id": 1
  },
  {
    "name": "Aleppo",
    "id": 2
  }
]
```

Get Neighborhoods by City

Endpoint: GET /api/city/{city_id}/neighborhoods

Authentication: Not Required

Success Response (200):

json

```
[
  {
    "name": "Al-MazzeH",
    "id": 1,
    "city_id": 1
  },
  {
    "name": "Kafr Sousa",
    "id": 2,
    "city_id": 1
  }
]
```

4. APARTMENT MANAGEMENT

Search Apartments with Filters

Endpoint: GET /api/apartments/index

Headers: Authorization: Bearer {token}

Query Parameters (all optional):

text

city_id: integer
neighborhood_id: integer
floor: integer
rooms: integer
bathrooms: integer
min_price: numeric
max_price: numeric
page: integer (default: 1)

Success Response (200): Paginated (12 items per page)

json

```
{
  "data": [
    {
      "id": 1,
      "title": "Luxury Apartment",
      "description": "Beautiful apartment...",
      "price": 150.00,
      "is_available": true,
      "approved_by_admin": false,
      "images": [
        {
          "id": 1,
          "image_path": "http://localhost/storage/apartments/filename.jpg",
          "is_main": true,
          "display_order": 0
        }
      ],
      "location": {
        "city": { "id": 1, "name": "Damascus" },
        "neighborhood": { "id": 1, "name": "Al-Mazzeah" },
      }
    }
  ]
}
```

```

        "floor": 3
    },
    "space": {
        "rooms": 2,
        "bathrooms": 1,
        "area": 120
    },
    "owner": {
        "profile": {
            "first_name": "John",
            "last_name": "Doe"
        }
    }
},
],
"links": {
    "first": "http://localhost/api/apartments/index?page=1",
    "last": "http://localhost/api/apartments/index?page=5",
    "prev": null,
    "next": "http://localhost/api/apartments/index?page=2"
},
"meta": {
    "current_page": 1,
    "last_page": 5,
    "total": 50
}
}

```

Get Single Apartment Details

Endpoint: GET /api/apartments/{apartment_id}

Headers: Authorization: Bearer {token}

Success Response (200): Complete apartment details with all relationships

Error Response (404):

json

```

{
    "message": "Failed to found apartment",
    "error": "Apartment not found"
}

```

Create New Apartment

Endpoint: POST /api/apartments/add

Headers: Authorization: Bearer {token}

Content-Type: multipart/form-data

Required Fields:

text

title: string

description: string

price: numeric

city_id: integer

neighborhood_id: integer

floor: integer

rooms: integer

bathrooms: integer

area: numeric

images: array of files (at least 1 image)

Success Response (201):

json

```
{
  "message": "Apartment created successfully",
  "apartment": {
    "id": 1,
    "title": "Luxury Apartment",
    "owner_id": 1,
    "price": 150.00,
    "is_available": true,
    "approved_by_admin": false,
    "location": { ... },
    "space": { ... },
    "images": [ ... ]
  }
}
```

Update Apartment

Endpoint: POST /api/apartments/update/{apartment_id}

Headers: Authorization: Bearer {token}

Content-Type: multipart/form-data

All Fields Optional:

text

title: string

description: string

price: numeric

city_id: integer

neighborhood_id: integer

floor: integer

rooms: integer

bathrooms: integer

area: numeric

images: array of files

Success Response (200):

json

```
{
  "message": "Apartment updated successfully",
  "apartment": { ... }
}
```

Error Responses:

- **403:** Owner mismatch
- **404:** Apartment not found
- **500:** Update failed

Delete Apartment

Endpoint: DELETE /api/apartments/{apartment_id}

Headers: Authorization: Bearer {token}

Success Response (200):

json

```
{
```



```
"message": "Apartment deleted"
}
```

Get My Apartments (Owner)

Endpoint: GET /api/apartments/my-apartments

Headers: Authorization: Bearer {token}

Success Response (200): Array of apartments owned by current user

No Apartments Response:

json

```
{
  "message": "you don't have any apartment"
}
```

Delete Apartment Image

Endpoint: DELETE /api/apartments/{apartment_id}/image/{image_id}

Headers: Authorization: Bearer {token}

Success Response (200):

json

```
{
  "message": "Image deleted successfully"
}
```

Set Main Image

Endpoint: POST /api/apartments/{apartment_id}/image/{image_id}/set-main

Headers: Authorization: Bearer {token}

Success Response (200):

json

```
{
  "message": "Main image updated successfully"
}
```

5. BOOKING SYSTEM

Create Booking Request

Endpoint: POST /api/apartment/{apartment_id}/book

Headers: Authorization: Bearer {token}

Content-Type: application/json

Request Body:

json

```
{
  "start_date": "2024-12-15",
  "end_date": "2024-12-20"
}
```

Success Response (201):

json

```
{
  "message": "Booking created and awaiting owner approval",
  "booking": {
    "id": 1,
    "apartment_id": 1,
    "user_id": 1,
    "start_date": "2024-12-15",
    "end_date": "2024-12-20",
    "total_price": 750.00,
    "status": "pending"
  }
}
```

Error Responses:

- **400:** Cannot book own apartment
- **409:** Date conflict exists

Get Tenant's Bookings

Endpoint: GET /api/my-bookings

Headers: Authorization: Bearer {token}

Success Response (200): Bookings grouped by status

json

```
{
  "pending": [ ... ],
  "approved": [ ... ],
  "rejected": [ ... ],
  "cancelled": [ ... ],
  "completed": [ ... ]
}
```

Get Owner's Bookings

Endpoint: GET /api/owner/bookings

Headers: Authorization: Bearer {token}

Returns: All bookings for owner's apartments

Get Booking Details

Endpoint: GET /api/bookings/{booking_id}

Headers: Authorization: Bearer {token}

Returns: Complete booking details with relationships

Accept Booking (Owner)

Endpoint: POST /api/booking/{booking_id}/accept

Headers: Authorization: Bearer {token}

Success Response (200):

json

```
{
  "message": "Booking accepted",
  "booking": { ... }
}
```

Reject Booking (Owner)

Endpoint: POST /api/booking/{booking_id}/reject

Headers: Authorization: Bearer {token}

Success Response (200):

json

```
{
  "message": "Booking rejected",
  "booking": { ... }
}
```

Request Cancellation (Tenant)

Endpoint: POST /api/booking/{booking_id}/cancel

Headers: Authorization: Bearer {token}

Success Response (202):

json

```
{
  "message": "Cancellation request submitted",
  "modification": {
    "id": 1,
    "booking_id": 1,
    "request_type": "cancel",
    "request_status": "pending"
  }
}
```

6. BOOKING MODIFICATIONS

Create Modification Request

Endpoint: POST /api/bookings/{booking_id}/modifications

Headers: Authorization: Bearer {token}

Content-Type: application/json

Request Body:

json

```
{
  "request_type": "cancel|extend|change_date",
  "start_requested_date": "2024-12-15", // required for change_date
  "end_requested_date": "2024-12-25" // required for extend & change_date
}
```

Success Response (201):

json

```
{
  "message": "Modification request created",
  "modification": {
    "id": 1,
    "booking_id": 1,
    "request_type": "extend",
    "request_status": "pending",
    "start_requested_date": "2024-12-15",
    "end_requested_date": "2024-12-25"
  }
}
```

Get Owner's Modification Requests

Endpoint: GET /api/owner/modifications

Headers: Authorization: Bearer {token}

Returns: Array of pending modification requests

Respond to Modification Request

Endpoint: POST /api/modifications/{modification_id}/respond

Headers: Authorization: Bearer {token}

Content-Type: application/json

Request Body:

json

```
{
  "request_status": "approved|rejected",
  "owner_response": "Optional explanation text"
}
```

Success Response (200):

json

```
{
  "message": "Modification handled successfully",
  "modification": { ... }
}
```

Get Modification Details

Endpoint: GET /api/modifications/{modification_id}

Headers: Authorization: Bearer {token}

Returns: Complete modification details

7. ADMIN FUNCTIONS

Get Pending Users

Endpoint: GET /api/pending-users

Authentication: Required

Returns: Array of users awaiting approval

Approve User

Endpoint: POST /api/approve/{user_id}

Authentication: Required

Success Response (200):

json

```
{
  "message": "User approved successfully",
  "user": { ... }
}
```

Delete User

Endpoint: DELETE /api/users/{user_id}

Authentication: Required

Success Response (200):

json

```
{  
  "message": "User deleted"  
}
```